
Building an HTTP Server in x86-64 Assembly

A line-by-line walkthrough of a raw-syscall web server

abouelmouahib Mehdi

July 2026

Introduction

This document explains a working HTTP server written entirely in x86-64 Linux assembly, using nothing but raw system calls — no C standard library, no external dependencies, not even a `main` function. It builds a program up from a bare `exit` all the way to a concurrent server that handles both GET and POST requests.

The central idea is that a web server is not magic. Underneath every framework and every library sits a small, fixed sequence of kernel operations: create a socket, bind it to an address, listen, accept a connection, read bytes, write bytes, close. Doing this in assembly strips away every abstraction and forces you to talk to the Linux kernel directly through the system-call interface. What follows is a tour of that conversation.

Throughout, the code uses **Intel syntax** (`mov dst, src`) and the **x86-64 Linux system-call convention**, which is the backbone of everything else, so it is worth stating up front.

How a system call works on x86-64

On 64-bit Linux, you ask the kernel to do something by loading a *syscall number* into the `rax` register, placing the arguments in a fixed order of registers, and executing the `syscall` instruction. The argument registers, in order, are `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`. The return value comes back in `rax` (a negative value indicates an error).

The relevant syscall numbers for this server are:

Number	Name	Purpose
0	<code>read</code>	read bytes from a file descriptor
1	<code>write</code>	write bytes to a file descriptor
2	<code>open</code>	open a file, returns a descriptor
3	<code>close</code>	close a descriptor
41	<code>socket</code>	create a network socket
43	<code>accept</code>	accept a pending connection
49	<code>bind</code>	attach a socket to an address/port
50	<code>listen</code>	mark a socket as passive
57	<code>fork</code>	duplicate the current process
60	<code>exit</code>	terminate the process

Everything the server does is some combination of these ten calls. The whole program is essentially a script that drives the kernel through them in the right order.

A quick note on an important detail: constants like `AF_INET` or `SOCK_STREAM` do not exist at the assembly level. They are C preprocessor names. To use them you must find the integer they resolve to (for example by grepping `/usr/include`), and then hard-code that integer. That is why the code below is full of bare numbers like 2 and 1 where a C program would read `AF_INET` and `SOCK_STREAM`.

The lifecycle of the server

At a high level the program follows a fixed arc: it opens a listening socket, then loops forever accepting clients. For each client it forks a child; the child figures out whether the request is a GET or a POST, does the corresponding file operation, sends a response, and exits, while the parent goes straight back to waiting for the next connection.

The sections below follow the source file `src/server.s` top to bottom.

Step 1 — Creating the socket

```
mov rdi, 2      # AF_INET (IPv4)
mov rsi, 1      # SOCK_STREAM (TCP)
mov rdx, 0      # protocol 0 (default = TCP for stream)
mov rax, 41     # socket
syscall
mov r8, rax     # save the listening socket fd
```

This is `socket(AF_INET, SOCK_STREAM, 0)`. `AF_INET` (value 2) selects the IPv4 address family; `SOCK_STREAM` (value 1) selects a reliable, connection-oriented byte stream, which is TCP. The protocol argument of 0 tells the kernel to pick the natural default for that type, again TCP.

The kernel returns a *file descriptor* — a small integer that from now on names this socket. The code stashes it in `r8`. A key insight of Unix networking is visible here: a socket is just a file descriptor, and the same `read/write/close` calls that work on files will work on it.

Step 2 — Binding to an address and port

```
mov rdi, r8      # the socket fd
lea rsi, [rip + sockaddr_in] # pointer to the address structure
mov rdx, 16      # sizeof(struct sockaddr_in)
mov rax, 49      # bind
syscall
```

`bind` attaches the socket to a concrete IP address and port so clients know where to reach it. The interesting part is the second argument: a pointer to a 16-byte `struct sockaddr_in`. Rather than building it on the stack, this server declares it as static data:

```
sockaddr_in:
    .word 2      # sin_family = AF_INET
    .word 0x901F # sin_port   = 8080, in network byte order
    .long 0      # sin_addr   = 0.0.0.0 (all interfaces)
    .quad 0      # 8 bytes of padding
```

The layout matches what the kernel expects, field by field. The first two bytes are the address family (2 = `AF_INET`). The next two are the port — and this is the subtle point the lab stresses. Ports travel in **network byte order**, which is big-endian: the most significant byte comes first. But x86 stores multi-byte values **little-endian** in memory. So the assembler directive `.word 0x901F` writes the bytes 1F 90 into memory, and `0x1F90 = 8080`, giving port 8080 correctly encoded on the wire.

(In the same way, `.word 0x5000` would encode port 80.) The four zero bytes for the address mean 0.0.0.0 — accept connections on any of the machine’s interfaces — and the final eight bytes are structure padding the kernel ignores.

Step 3 — Listening

```
mov rdi, r8      # the socket fd
mov rsi, 0       # backlog
mov rax, 50      # listen
syscall
```

`listen` flips the socket from an *active* socket (one you would use to connect out) into a *passive* one that waits for inbound connections. The second argument is the backlog: how many completed-but-not-yet-accepted connections the kernel will queue. After this call the socket is ready, but nothing has been accepted yet — the server is now advertising itself on port 8080.

Step 4 — The accept loop

```
accept_loop:

mov rdi, r8      # listening socket
mov rsi, 0       # don't care about client address
mov rdx, 0
mov rax, 43      # accept
syscall
mov r9, rax      # r9 = fresh per-client socket fd
```

This is where the server begins its real work, and it is a loop it will never leave on its own. `accept` **blocks** until a client connects. When one does, it returns a brand-new file descriptor dedicated to that single conversation. This is a crucial distinction: `r8` remains the listening socket (used only to accept more clients), while `r9` is the connected socket used to actually talk to *this* client. Passing NULL (zero) for the address arguments simply says the server does not care to record who connected.

Step 5 — Forking for concurrency

```
mov rax, 57      # fork
syscall

cmp rax, 0
je child_process # rax == 0 -> we are the child

# ---- parent path ----
mov rdi, r9      # close our copy of the client socket
mov rax, 3       # close
```

```
syscall
jmp accept_loop      # go wait for the next client
```

`fork` is the heart of the concurrency model, and it behaves in a way that surprises newcomers: it returns **twice**. The kernel duplicates the calling process, producing a near-identical child. In the parent, `fork` returns the child's process ID (a positive number); in the child, it returns 0. Both processes then continue executing from the same instruction, and the `cmp rax, 0` is what lets each one discover which role it plays.

The design choice here is the classic *fork-per-connection* server:

- The **parent** does not talk to the client at all. It closes its own copy of the client descriptor `r9` (the child has its own copy, so the connection stays alive) and jumps straight back to `accept_loop` to wait for the next client. This is what makes the server concurrent — the parent is almost always sitting in `accept`, ready for someone new, no matter how slow the current client is.
- The **child** falls through to `child_process` and handles this one connection to completion, then exits.

Because each connection gets its own process with its own memory, two clients being served at the same time cannot interfere with each other's buffers or registers.

```
child_process:

mov rdi, r8          # child closes the LISTENING socket
mov rax, 3           # close
syscall
```

The first thing the child does is close the listening socket. The child has no business accepting new connections — that is the parent's job — and closing the inherited copy is good hygiene, preventing descriptor leaks.

Step 6 — Reading the request

```
mov rdi, r9          # client socket
lea rsi, [rip + http_request] # buffer
mov rdx, 4096        # buffer size
mov rax, 0           # read
syscall
mov r14, rax         # r14 = number of bytes actually read
```

The child reads up to 4096 bytes of the raw HTTP request into a buffer in the `.bss` section. The return value — the actual byte count — is saved in `r14`, because it will be needed later to compute the length of a POST body. An HTTP request the server might receive looks like this:

```
GET /hello.txt HTTP/1.0\r\n
Host: 127.0.0.1:8080\r\n
\r\n
```

or, for a POST:

```
POST /upload.txt HTTP/1.0\r\n
Content-Length: 24\r\n
\r\n
Data written by Assembly
```

Everything the server needs is text it must parse out of that buffer by hand.

Step 7 — Parsing the requested path

The request line always has the shape `METHOD <space> /path <space> HTTP/1.0`. The path is the text between the first and second spaces. The code finds it in two passes.

```
xor rcx, rcx                # rcx = scan index into the request
find_space_loop:
    cmp BYTE PTR [http_request + rcx], ' '
    je start_copy
    inc rcx
    jmp find_space_loop
```

This scans forward until it hits the first space (the one right after `GET` or `POST`).

```
start_copy:
    inc rcx                # skip the space...
    inc rcx                # ...and skip the leading '/'
    xor r10, r10           # r10 = index into the destination
    lea r11, [rip + path_file]
copy_loop:
    mov al, BYTE PTR [http_request + rcx]
    cmp al, ' '
    je finish_copy
    mov BYTE PTR [r11 + r10], al # copy one character of the path
    inc r10
    inc rcx
    jmp copy_loop

finish_copy:
    mov BYTE PTR [r11 + r10], 0 # NUL-terminate the filename
```

After the first space it advances two bytes — one past the space, one past the leading `/` — so that a request for `/hello.txt` yields the filename `hello.txt` (a relative path, which is what `open` will want). It then copies characters into the `path_file` buffer until it meets the next space, and writes a terminating zero byte so the string can be handed to `open`.

Step 8 — Branching on the HTTP method

```
mov al, 'G'
cmp BYTE PTR [http_request], al
je GET_request
# otherwise, fall through into the POST handling
```

The method is decided by a single character: the very first byte of the request. If it is `G`, this is a `GET`, and control jumps to the `GET_request` block. Otherwise the code assumes a `POST` and falls through into the `POST`-handling logic. This is deliberately minimal — it does not distinguish `PUT`, `DELETE`, or anything else — but for the two methods this server supports, one byte is enough.

Step 9 — Handling POST (writing to a file)

A `POST` carries a *body* — the data to be stored — after the headers. HTTP separates headers from body with a blank line, i.e. the four-byte sequence `\r\n\r\n`. The code finds that boundary:

```
xor rcx, rcx
find_headers_end:
    cmp DWORD PTR [http_request + rcx], 0x0A0D0A0D    # "\r\n\r\n"
    je found_headers_end
    inc rcx
    jmp find_headers_end
```

It compares four bytes at a time against the constant `0x0A0D0A0D`. Reading that constant on a little-endian machine, the lowest byte comes first in memory, so the bytes in memory are `0D 0A 0D 0A` — exactly `\r\n\r\n`. This is the same little-endian reasoning as the port number, applied to pattern matching.

```
found_headers_end:
    add rcx, 4          # step past the "\r\n\r\n"
    mov r13, r14        # r14 = total bytes read
    sub r13, rcx        # r13 = body length = total - header length
    mov rbx, r13        # keep a copy of the body length
```

Rather than parse the `Content-Length` header (the “obvious” but fiddlier route), the server computes the body length arithmetically: total bytes read minus the number of bytes up to and including the header terminator. This is the elegant trick — the length simply falls out of subtraction.

```
xor r10, r10
lea r11, [rip + flag]
copy_body:
    mov al, BYTE PTR [http_request + rcx]
    mov BYTE PTR [r11 + r10], al
    inc r10
    dec r13
    inc rcx
    cmp r13, 0
    jne copy_body
```

The body is copied out of the request buffer into a separate buffer (named `flag` in the source). Then it is written to disk:

```
lea rdi, [rip + path_file]
mov rsi, 65          # O_WRONLY | O_CREAT
mov rdx, 0777        # mode if the file is created
```

```

mov rax, 2          # open
syscall
mov r8, rax         # reuse r8 for the file fd

mov rdi, r8
lea rsi, [rip + flag]
mov rdx, rbx        # body length
mov rax, 1          # write
syscall

mov rdi, r8
mov rax, 3          # close
syscall

```

open is called with flags 65, which is `O_WRONLY | O_CREAT` (write-only, create if missing), and mode 0777. The body is written into the file, and the file is closed. Finally the server acknowledges success to the client:

```

mov rdi, r9
lea rsi, [rip + http_response]
mov rdx, 19         # length of "HTTP/1.0 200 OK\r\n\r\n"
mov rax, 1          # write
syscall
jmp fin

```

The response is the fixed 19-byte string `HTTP/1.0 200 OK\r\n\r\n`. A POST returns only this status line — no body — telling the client the write succeeded.

Step 10 — Handling GET (reading a file)

```

GET_request:
    lea rdi, [rip + path_file]
    mov rsi, 0        # O_RDONLY
    mov rdx, 0
    mov rax, 2        # open
    syscall
    mov r10, rax      # file fd

    mov rdi, r10
    lea rsi, [rip + flag]
    mov rdx, 256      # read up to 256 bytes
    mov rax, 0        # read
    syscall
    mov r12, rax      # r12 = bytes read from the file

    mov rdi, r10
    mov rax, 3        # close

```



```
syscall
```

The GET path is the mirror image of POST. It opens the requested file read-only (`O_RDONLY = 0`), reads up to 256 bytes of its contents into the `flag` buffer, remembers how many bytes came back in `r12`, and closes the file.

```
mov rdi, r9
lea rsi, [rip + http_response]
mov rdx, 19          # status line
mov rax, 1
syscall

mov rdi, r9
lea rsi, [rip + flag]
mov rdx, r12         # exactly the number of bytes read
mov rax, 1           # write
syscall
```

Then it writes two things to the client socket: first the same 19-byte 200 OK status line, then the file's contents. Note that the body length sent is `r12`, the exact number of bytes read, so the client receives precisely the file and nothing more.

Step 11 — Finishing the connection

```
fin:
    mov rdi, r9
    mov rax, 3          # close the client socket
    syscall

    mov rdi, 0
    mov rax, 60         # exit(0)
    syscall
```

Both paths converge at `fin`. The child closes the client socket — which is what actually signals end-of-response to the client — and then calls `exit(0)`. This is important: **only the child exits**. The parent never reaches this code; it is off in `accept_loop`. So each connection spins up a short-lived process that does one job and dies, while the long-lived parent keeps the service alive.

A subtle real-world caveat: because the parent never calls `wait`, finished children become *zombie* processes until the parent dies. For an educational server this is harmless, but a production server would reap its children (or ignore `SIGCHLD`) to avoid accumulating them.

Memory layout

The data the program relies on lives in two sections:

```
.section .data
sockaddr_in:          # the 16-byte bind address, described above
```

```

...
http_response: .ascii "HTTP/1.0 200 OK\r\n\r\n"

.section .bss
http_request: .skip 4096    # inbound request buffer
path_file:    .skip 256     # parsed filename
flag:         .skip 256     # file contents / POST body

```

The `.data` section holds initialized constants: the `sockaddr_in` structure and the canned response string. The `.bss` section holds zero-initialized scratch buffers that exist only at runtime — the request buffer, the parsed path, and the file/body buffer. Splitting them this way keeps the constants in the binary while letting the kernel allocate the buffers as zeroed pages on load.

Building and running

The `Makefile` assembles and links with the GNU toolchain, deliberately avoiding the C runtime:

```

build/server.o: src/server.s
    as --64 src/server.s -o build/server.o
build/http-server: build/server.o
    ld build/server.o -o build/http-server

```

`as` produces an object file and `ld` links it directly. Because the program defines `_start` (not `main`) and makes only raw syscalls, no `libc` is linked in — the resulting binary is tiny and self-contained. Running it starts the listener on port 8080; `curl http://127.0.0.1:8080/hello.txt` exercises the GET path and `curl -X POST ../upload.txt --data "..."` exercises POST.

Summary of the key principles

The server rests on a handful of ideas that generalize far beyond this one program. A socket is just a file descriptor, so the ordinary `read/write/close` calls serve the network as well as the disk. Network byte order is big-endian while x86 memory is little-endian, which is why the port constant looks “backwards” in the source. The listening socket and the per-connection socket are different descriptors with different jobs. Concurrency comes from `fork`, which returns twice — zero in the child, the child’s PID in the parent — letting one call split the program into a listener and a worker. HTTP itself is plain text terminated by `\r\n\r\n`, so parsing it is a matter of scanning bytes for spaces and that blank-line boundary. And the method is decided by a single leading character, `G` for GET or otherwise POST.

Put together, these give a complete, if minimal, web server in a couple hundred lines of assembly
